

Switching Circuits and Logic Design (CS21202)

Module 8

Synthesis and Minimization of Finite State Machines

Prof. Indranil Sengupta

Department of Computer Science and Engineering

IIT Kharagpur

1

Introduction

- Combinational and Sequential Circuits
 - In a *combinational circuit*, the outputs depend only on the applied input values and not on the past history.
 - In a *sequential circuit*, the outputs depend not only on the applied input values but also on the internal state.
 - The internal states also change with time.
 - The number of states is finite, and hence a sequential circuit is also referred to as a *Finite State Machine (FSM)*.
- Most of the practical circuits are sequential in nature.

Switching Circuits and Logic Design (CS20202)

2

2

How are the States Stored and Changed?

- In a sequential circuit, we need some mechanism to store the internal states.
 - We use flip-flops or latches to store the internal states.
 - If we use k flip-flops, there can be a maximum of 2^k internal states.
- Depending upon the way the states change, sequential circuits can be categorized into two types:
 - a) **Synchronous**: The states change in synchronism with a clock pulse.
 - b) **Asynchronous**: There is no clock pulse for synchronism; all state changes occur depending on the delays of the circuit elements (e.g. gates).

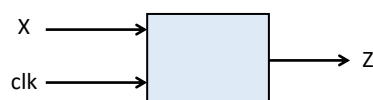
Switching Circuits and Logic Design (CS20202)

3

3

Finite State Machine (FSM)

- A FSM can be represented either in the form of a **state table** or in the form of a **state transition diagram**.
 - Variations exist, e.g. Algorithmic State Machine (ASM) chart.
- Example:
 - A circuit to detect 3 or more 1's in a serial bit stream.
 - The bits are applied serially in synchronism with a clock.
 - The output will become 1 whenever it detects 3 or more consecutive 1's in the stream.



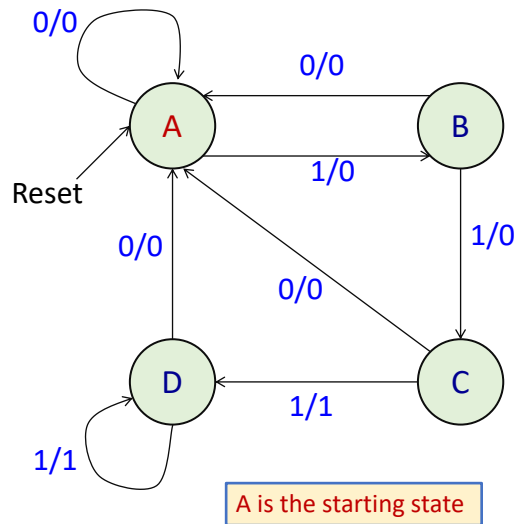
Switching Circuits and Logic Design (CS20202)

4

4

State Table

Reset	PS	Input	NS	Output
1	-	-	A	0
0	A	0	A	0
0	A	1	B	0
0	B	0	A	0
0	B	1	C	0
0	C	0	A	0
0	C	1	D	1
0	D	0	A	0
0	D	1	D	1

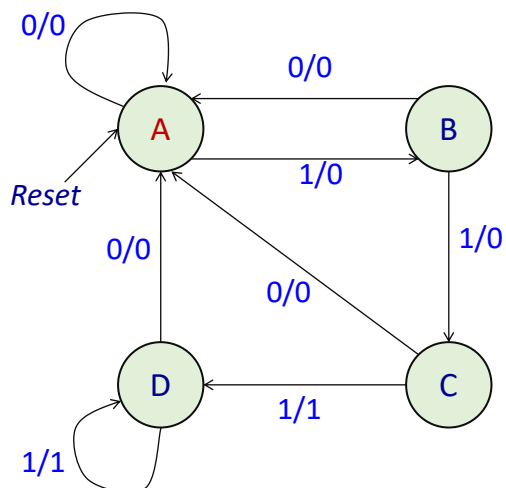
State Transition Diagram

Switching Circuits and Logic Design (CS20202)

5

5

More Formal Way to Write State Table



PS	NS, Z			
	RX = 00	RX = 01	RX = 10	RX = 11
A	A,0	B,0	A,0	A,0
B	A,0	C,0	A,0	A,0
C	A,0	D,1	A,0	A,0
D	A,0	D,1	A,0	A,0

R: Reset
X: Input

Switching Circuits and Logic Design (CS20202)

6

6

Mealy and Moore FSM Types

- A deterministic FSM can be mathematically defined as a 5-tuple

$$M = (\Sigma, \Gamma, S, s_0, \delta, \lambda)$$

where Σ is the set of input combinations, Γ is the set of output combinations, S is a finite set of states, $s_0 \in S$ is the initial state, δ is the state-transition function, and λ is the output function.

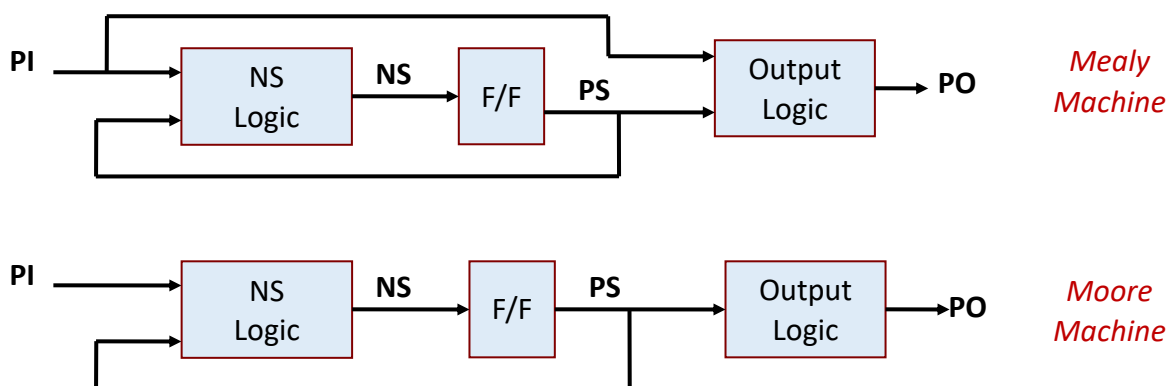
- Here, $\delta : S \times \Sigma \rightarrow S$
 - Present state* (PS) and *present input* determines the *next state* (NS).
- For Mealy machine, $\lambda : S \times \Sigma \rightarrow \Gamma$ (output depends on state + inputs)
- For Moore machine, $\lambda : S \rightarrow \Gamma$ (output depends only on the state)

Switching Circuits and Logic Design (CS20202)

7

7

Pictorial Depiction

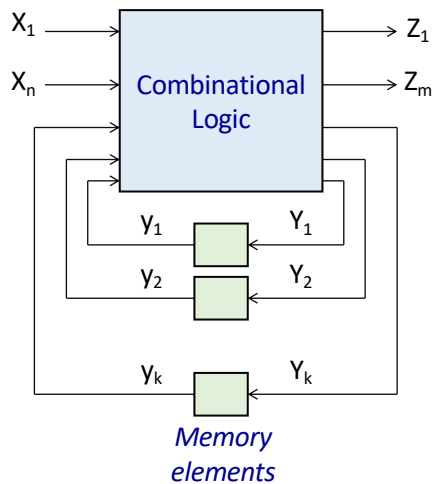


Switching Circuits and Logic Design (CS20202)

8

8

Model of Synchronous Sequential Machines



Input variables $X = \{X_1, X_2, \dots, X_n\}$

Output variables $Z = \{Z_1, Z_2, \dots, Z_m\}$

State variables $Y = \{y_1, y_2, \dots, y_k\}$

Input alphabet $\Sigma :: \text{set of } 2^n \text{ input patterns}$

Output alphabet $\Gamma :: \text{set of } 2^m \text{ output patterns}$

States $S :: \text{set of } 2^k \text{ k-tuples}$

Present State $:: \{y_1, y_2, \dots, y_k\}$

Next State $:: \{Y_1, Y_2, \dots, Y_k\}$

When clock comes, Next State gets copied to Present State.

Switching Circuits and Logic Design (CS20202)

9

9

Synthesis of Synchronous Sequential Circuits

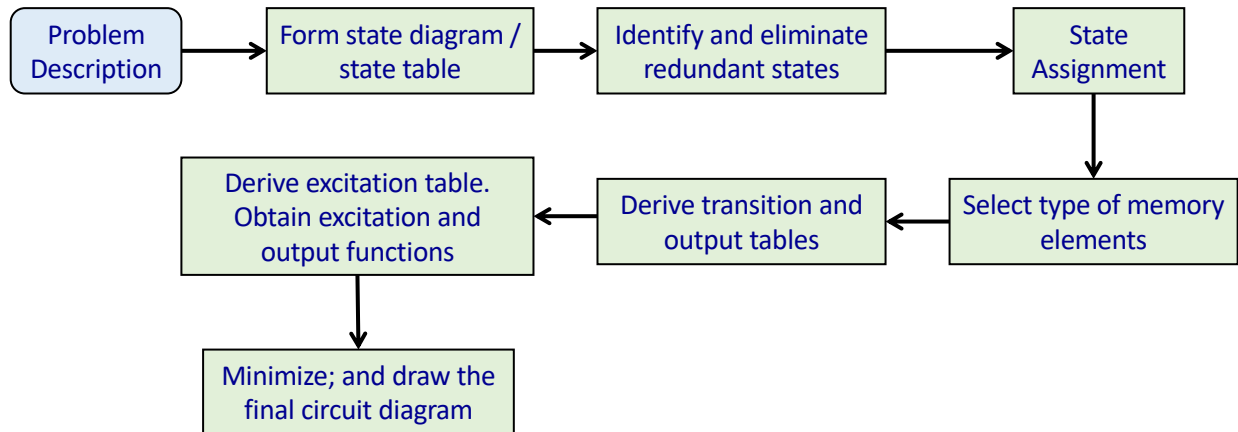
1. From a description of the problem, form a *state diagram* or *state table*.
2. Check whether the table contains any *redundant states*; if so, remove them.
 - To be discussed later.
3. Select a *state assignment* and determine the type of *memory elements*.
 - We can choose any type of memory elements (flip-flops) to maintain states.
 - Final circuit complexity shall depend on this selection.
4. Derive *transition and output tables*.
5. Derive the *excitation table*, and obtain *excitation and output functions*.
6. Minimize the functions, and obtain the *circuit diagram*.

Switching Circuits and Logic Design (CS20202)

10

10

The Overall Flow Diagram



Switching Circuits and Logic Design (CS20202)

11

11

State Transition Diagram and State Table

- A *state transition diagram* specifies the different states of a FSM, the conditions under which state changes occur, and the corresponding outputs.
 - States are denoted as circles, and labeled with unique symbols.
 - Transitions are represented as directed arrows between pair of states.
 - Each transition is labeled by α/β , where α denotes an input combination and β denotes an output combination.
- A state table is an alternate depiction of the state transition diagram.
 - For every value of the present state, it specifies the next state and the output for every input combination.

Switching Circuits and Logic Design (CS20202)

12

12

Example 1

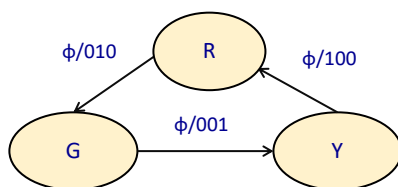
- There are three lamps, RED, GREEN and YELLOW, that should glow cyclically with a fixed time interval (say, 1 second).
- Some observations:
 - The FSM will have three states, corresponding to the glowing state of the lamps.
 - The input set is null; state transition will occur whenever clock signal comes.
 - This is a *Moore Machine*, since the lamp that will glow only depends on the state and not on the inputs (here null).



Switching Circuits and Logic Design (CS20202)

13

13



State Transition Diagram

PS	NS, Z
RED	GREEN, 010
GREEN	YELLOW, 001
YELLOW	RED, 100

State Table

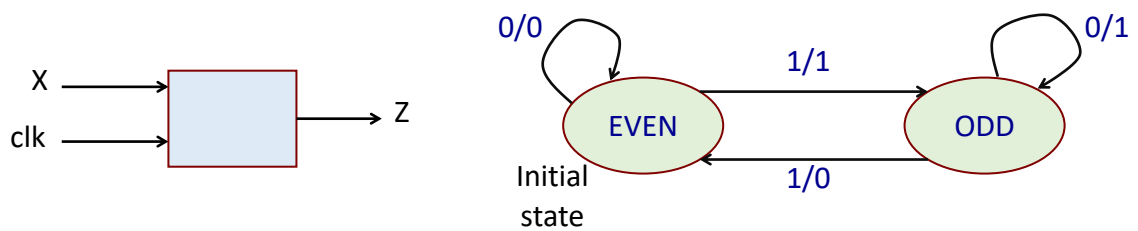
Switching Circuits and Logic Design (CS20202)

14

14

Example 2

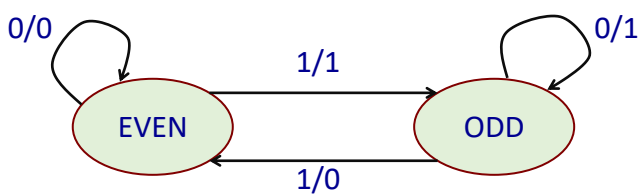
- Design of a serial parity detector.
 - A continuous stream of bits is fed to a circuit in synchronism with a clock. The circuit will be generating a bit stream as output, where a 0 will indicate “even number of 1’s seen so far” and a 1 will indicate “odd number of 1’s seen so far”.
 - This is also a *Moore Machine*.



Switching Circuits and Logic Design (CS20202)

15

15



State Transition Diagram

PS	NS, Z	
	$X = 0$	$X = 1$
EVEN	EVEN, 0	ODD, 1
ODD	ODD, 1	EVEN, 0

State Table

Switching Circuits and Logic Design (CS20202)

16

16

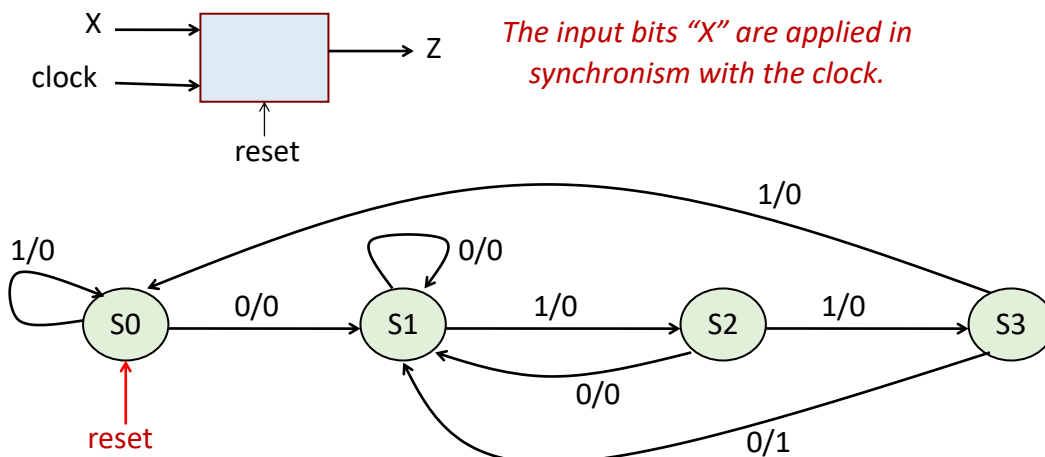
Example 3

- Design of a sequence detector.
 - A circuit accepts a serial bit stream “X” as input and produces a serial bit stream “Z” as output.
 - Whenever the bit pattern “0110” appears in the input stream, it outputs Z = 1; at all other times, Z = 0.
 - Overlapping occurrences of the pattern are also detected.
 - This is a *Mealy Machine*.
 - Example: x :- 0 1 0 1 0 1 1 0 1 1 0 0 1 0 1 1 0 1 1 1 0
 z :- 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0

Switching Circuits and Logic Design (CS20202)

17

17



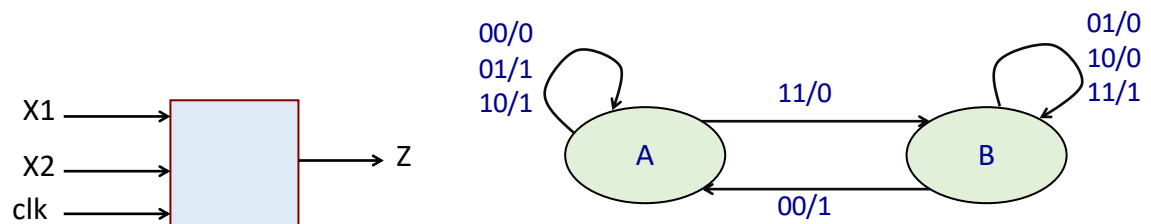
Switching Circuits and Logic Design (CS20202)

18

18

Example 4

- A 2-bit serial adder which takes two serial bit streams X1 and X2 as inputs, and generates a serial bit stream Z as output.

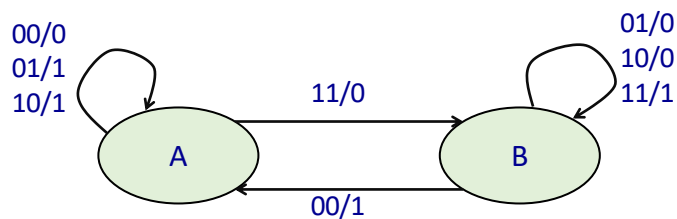


The state represents the carry; A means carry is 0, B means carry is 1.

Switching Circuits and Logic Design (CS20202)

19

19



State Transition Diagram

PS	NS, Z			
	X = 00	X = 01	X = 10	X = 11
A	A, 0	A, 1	A, 1	B, 0
B	A, 1	B, 0	B, 0	B, 1

State Table

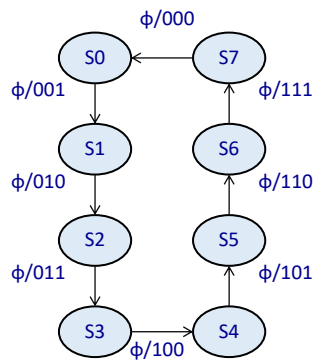
Switching Circuits and Logic Design (CS20202)

20

20

Example 5

- A 3-bit binary counter that counts in the sequence 000, 001, 010, 011, 100, 101, 110, 111, and then back again to 000.



PS	NS, Z
S0	S1, 001
S1	S2, 010
S2	S3, 011
S3	S4, 100
S4	S5, 101
S5	S6, 110
S6	S7, 111
S7	S0, 000

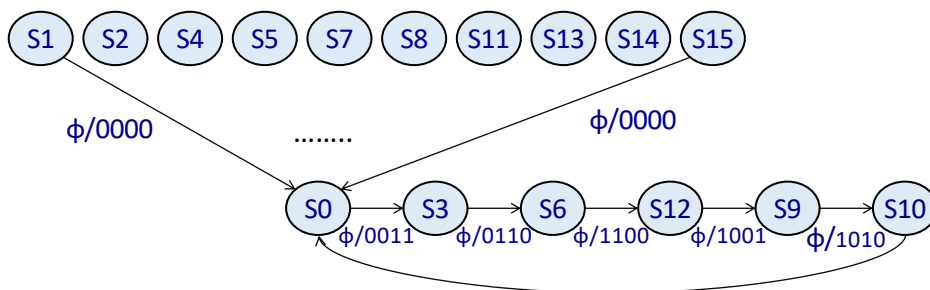
Switching Circuits and Logic Design (CS20202)

21

21

Example 6

- A 4-bit counter that counts in the sequence 0000, 0011, 0110, 1100, 1001, 1010, and then back again to 0000. If the counter starts with any of the other states, it will go to 0000 at the next clock.



Switching Circuits and Logic Design (CS20202)

22

22

FSM Synthesis

Switching Circuits and Logic Design (CS20202)

23

23

FSM Synthesis

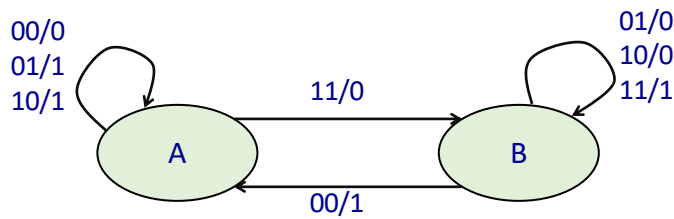
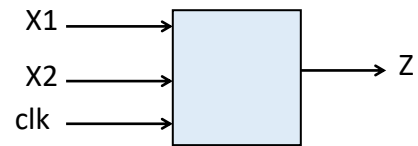
- We have seen how the state transition diagram / state table can be constructed from the problem description.
- To construct the circuit, the following further steps are required:
 - Assign unique binary code to each state: *state assignment*.
 - Construct the *transition/output table*.
 - Select *type of memory elements*: SR or JK or D or T, and construct *excitation table*.
 - Obtain the *excitation and output functions*, and minimize them.
 - Realize the functions using gates or any other combinational circuit modules.
- We shall illustrate the process with examples.

Switching Circuits and Logic Design (CS20202)

24

24

Example 1: Serial Adder



State Transition Diagram

PS	NS, Z			
	X = 00	X = 01	X = 10	X = 11
A	A, 0	A, 1	A, 1	B, 0
B	A, 1	B, 0	B, 0	B, 1

State Table

Switching Circuits and Logic Design (CS20202)

25

25

- State assignment:
 - Two states, and so one bit is sufficient.
 - Suppose we assign *0 for state A*, and *1 for state B*.

PS	NS, Z			
	X = 00	X = 01	X = 10	X = 11
0	0, 0	0, 1	0, 1	1, 0
1	0, 1	1, 0	1, 0	1, 1

After state assignment

	PS	NS				Output Z			
X ₁ X ₂		00	01	10	11	00	01	10	11
	0	0	0	0	1	0	1	1	0
	1	0	1	1	1	1	0	0	1

Transition/output Table

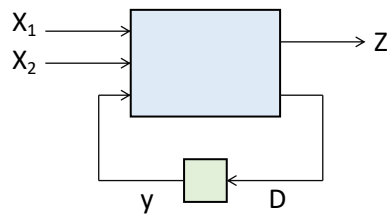
Switching Circuits and Logic Design (CS20202)

26

26

- Select memory element, and construct excitation/output function.

- Suppose we use *D flip-flop*.



PS	NS				Output Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Transition/output Table

y	D				Output Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Excitation and Output Function

Switching Circuits and Logic Design (CS20202)

27

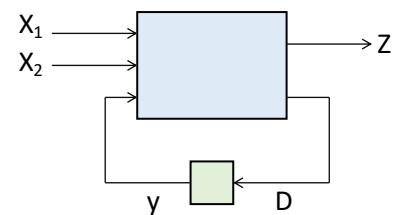
27

X_1X_2		00	01	11	10
y	0			1	
	1		1	1	1

$$D = X_1X_2 + X_1y + X_2y$$

X_1X_2		00	01	11	10
y	0		1		1
	1	1		1	

$$Z = X_1 \oplus X_2 \oplus y$$



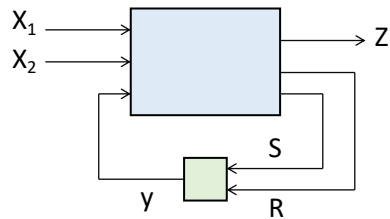
y	D				Output Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Switching Circuits and Logic Design (CS20202)

28

28

- Alternate design.
 - Suppose we use *SR flip-flop*



PS	NS				Output Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Transition/output Table

Circuit changes		Required value	
From	To	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

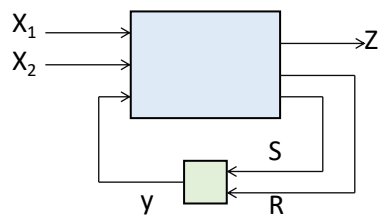
Excitation table of SR flip-flop

Switching Circuits and Logic Design (CS20202)

29

29

- Alternate design.
 - Suppose we use *SR flip-flop*



PS	NS				Output Z			
	00	01	10	11	00	01	10	11
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Transition/output Table

y	S R				Output Z			
	00	01	10	11	00	01	10	11
0	0X	0X	0X	10	0	1	1	0
1	01	X0	X0	X0	1	0	0	1

Excitation and Output Function

Switching Circuits and Logic Design (CS20202)

30

30

	X_1X_2			
y	00	01	11	10
0			1	
1		X	X	X

$S = X_1X_2$

	X_1X_2			
y	00	01	11	10
0		1		1
1	1		1	

$Z = X_1 \oplus X_2 \oplus y$

	X_1X_2			
y	00	01	11	10
0	X	X		X
1	1			

$R = X_1'X_2'$

y	S R				Output Z			
	00	01	10	11	00	01	10	11
0	0X	0X	0X	10	0	1	1	0
1	01	X0	X0	X0	1	0	0	1

Excitation and Output Function

Switching Circuits and Logic Design (CS20202) 31

31

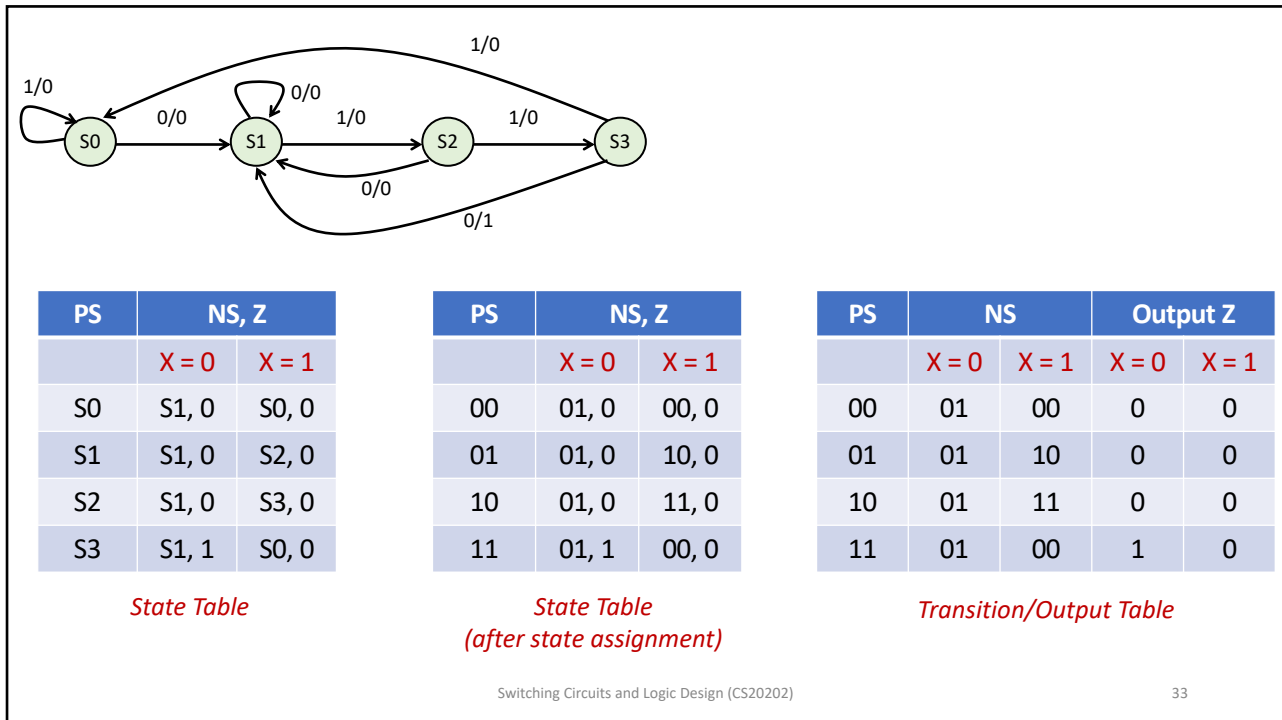
Example 2: Sequence Detector

- Design of a sequence detector.
 - A circuit accepts a serial bit stream "X" as input and produces a serial bit stream "Z" as output.
 - Whenever the bit pattern "0110" appears in the input stream, it outputs $Z = 1$; at all other times, $Z = 0$.
 - Overlapping occurrences of the pattern are also detected.



- Example: X :- 0 1 0 1 0 1 1 0 1 1 0 0 1 0 1 1 0 1 1 0
 Z :- 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0

32



33

- Suppose we use *T flip-flop*.
 - Two flip-flops, inputs T_1 and T_2 .

PS	NS		Output Z	
	X = 0	X = 1	X = 0	X = 1
00	01	00	0	0
01	01	10	0	0
10	01	11	0	0
11	01	00	1	0

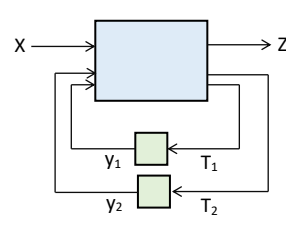
y_1y_2	$T_1 T_2$		Z	
	X = 0	X = 1	X = 0	X = 1
00	01	00	0	0
01	00	11	0	0
10	11	01	0	0
11	10	11	1	0

Transition/Output Table

Excitation and Output Function

34

$y_1 y_2$	$T_1 T_2$		Z	
	$X=0$	$X=1$	$X=0$	$X=1$
00	01	00	0	0
01	00	11	0	0
10	11	01	0	0
11	10	11	1	0



$X \backslash y_1 y_2$	00	01	11	10
0			1	1
1		1	1	

$T_1 = X' y_1 + X y_2$

$X \backslash y_1 y_2$	00	01	11	10
0	1			1
1		1	1	1

$T_2 = X' y_2' + y_1 y_2' + X y_2$

$X \backslash y_1 y_2$	00	01	11	10
0			1	
1				

$Z = X' y_1 y_2$

Switching Circuits and Logic Design (CS20202) 35

35

- Let us repeat the process using *J-K flip-flop*.

PS	NS		Output Z	
	$X=0$	$X=1$	$X=0$	$X=1$
00	01	00	0	0
01	01	10	0	0
10	01	11	0	0
11	01	00	1	0

Transition/Output Table

$y_1 y_2$	$J_1 K_1 J_2 K_2$		Z	
	$X=0$	$X=1$	$X=0$	$X=1$
00	0X 1X	0X 0X	0	0
01	0X X0	1X X1	0	0
10	X1 1X	X0 1X	0	0
11	X1 X0	X1 X1	1	0

Excitation and Output Function

36

$y_1 y_2$	$J_1 K_1$		$J_2 K_2$		Z	
	$X = 0$		$X = 1$		$X = 0$	$X = 1$
00	0X	1X	0X	0X	0	0
01	0X	X0	1X	X1	0	0
10	X1	1X	X0	1X	0	0
11	X1	X0	X1	X1	1	0

$y_1 y_2$	00	01	11	10
X			X	X
0			X	X
1		1	X	X

$J_1 = X y_1$

$y_1 y_2$	00	01	11	10
X				
0	X	X	1	1
1	X	X	1	

$K_1 = X' + X y_2$

Excitation and Output Function

$y_1 y_2$	00	01	11	10
X				
0	1	X	X	1
1		X	X	1

$J_2 = X' + y_1$

$y_1 y_2$	00	01	11	10
X				
0	X			X
1	X	1	1	X

$K_2 = X$

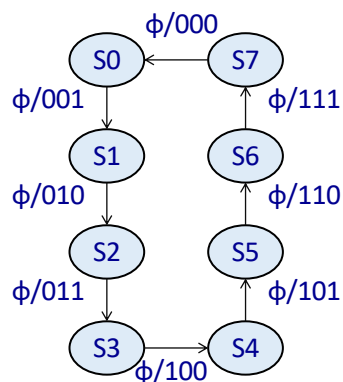
$y_1 y_2$	00	01	11	10
X				
0			1	
1				

$Z = X' y_1 y_2$

37

37

Example 3: Modulo-3 counter



PS	NS, Z
S0	S1, 001
S1	S2, 010
S2	S3, 011
S3	S4, 100
S4	S5, 101
S5	S6, 110
S6	S7, 111
S7	S0, 000

PS	NS, Z
000	001, 001
001	010, 010
010	011, 011
011	100, 100
100	101, 101
101	110, 110
110	111, 111
111	000, 000

Let us design *using T flip-flops*

38

38

$y_1 y_2 y_3$	$T_1 T_2 T_3$	Z
000	001	001
001	011	010
010	001	011
011	111	100
100	001	101
101	011	110
110	001	111
111	111	000

Excitation and Output Function

y_3	$y_1 y_2$	00	01	11	10
0			1	1	
1		1			1

$$Z_2 = y_2 y_3' + y_2' y_3$$

y_3	$y_1 y_2$	00	01	11	10
0					
1			1	1	

$$T_1 = y_2 y_3$$

y_3	$y_1 y_2$	00	01	11	10
0		1	1	1	1
1		1	1	1	1

$$T_3 = 1$$

y_3	$y_1 y_2$	00	01	11	10
0					
1		1	1	1	1

$$T_2 = y_3$$

y_3	$y_1 y_2$	00	01	11	10
0				1	1
1			1		1

$$Z_1 = y_1 y_3' + y_1 y_2' + y_1' y_2' y_3$$

y_3	$y_1 y_2$	00	01	11	10
0		1	1	1	1
1					

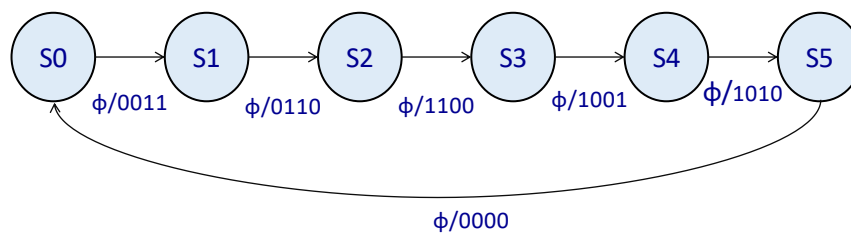
$$Z_3 = y_3'$$

39

39

Example 4: A 4-bit counter

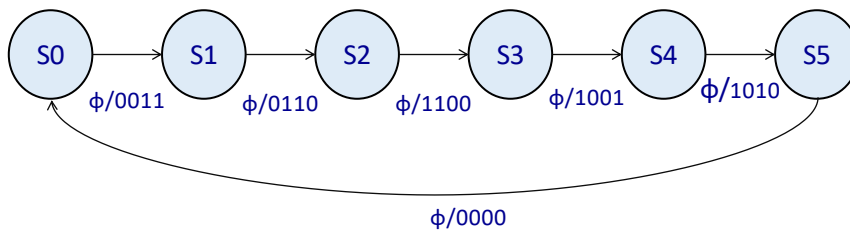
- Design 4-bit counter that counts in the sequence 0000, 0011, 0110, 1100, 1001, 1010, and then back again to 0000.



6 states → We need 3 state variables

40

40



PS	NS, Z	PS	NS, Z
S0	S1, 0011	000	001, 0011
S1	S2, 0110	001	010, 0110
S2	S3, 1100	010	011, 1100
S3	S4, 1001	011	100, 1001
S4	S5, 1010	100	101, 1010
S5	S0, 0000	101	000, 0000

41

41

Alternative 1: Design using T flip-flops

PS	NS, Z	$y_1 y_2 y_3$	$T_1 T_2 T_3$	Z
000	001, 0011	0 0 0	0 0 1	0 0 1 1
001	010, 0110	0 0 1	0 1 1	0 1 1 0
010	011, 1100	0 1 0	0 0 1	1 1 0 0
011	100, 1001	0 1 1	1 1 1	1 0 0 1
100	101, 1010	1 0 0	0 0 1	1 0 1 1
101	000, 0000	1 0 1	1 0 1	0 0 0 0

$y_1 y_2$		00	01	11	10
y_3	0			X	
	1		1	X	1

$$T_1 = y_2 y_3 + y_1 y_3$$

$y_1 y_2$		00	01	11	10
y_3	0			X	
	1	1	1	X	

$$T_2 = y_1' y_3$$

$y_1 y_2$		00	01	11	10
y_3	0	1	1	X	1
	1	1	1	X	1

$$T_3 = 1$$

42

42

$y_3 \backslash y_1 y_2$	00	01	11	10
0		1	X	1
1		1	X	

$Z_1 = y_2 + y_1 y_3'$

$y_3 \backslash y_1 y_2$	00	01	11	10
0		1	X	
1	1		X	

$Z_2 = y_2 y_3' + y_1' y_2' y_3$

$y_3 \backslash y_1 y_2$	00	01	11	10
0	1	1	X	1
1			X	

$Z_3 = y_3'$

$y_3 \backslash y_1 y_2$	00	01	11	10
0	1		X	1
1		1	X	

$Z_4 = y_2' y_3' + y_2 y_3$

$y_1 y_2 y_3$	$T_1 T_2 T_3$	Z
000	001	0011
001	011	0110
010	001	1100
011	111	1001
100	001	1011
101	101	0000

$Z = (Z_1 Z_2 Z_3 Z_4)$

43

Alternative 2: Design using S-R flip-flops

PS	NS, Z	$y_1 y_2 y_3$	$S_1 R_1 \ S_2 R_2 \ S_3 R_3$	Z
000	001, 0011	000	0X 0X 10	0011
001	010, 0110	001	0X 10 01	0110
010	011, 1100	010	0X X0. 10	1100
011	100, 1001	011	10 01. 01	1001
100	101, 1010	100	X0 0X 10	1011
101	110, 0000	101	X0 10. 01	0000

We have to minimize six functions for S1, R1, S2, R2, S3, R3 and four functions for Z.

44

Minimization of Finite State Machines

Switching Circuits and Logic Design (CS20202)

45

45

Basic Concept

- Simplify the specification of an FSM so that the final circuit realization becomes cheaper.
- Some states in the state transition diagram or state table may be equivalent.
 - We need to identify the sets of equivalent states.
 - For each such set, we merge all its states into a single state.
 - Basically, we seek to eliminate the redundant states.

46

46

Elimination of Redundant States

- Basic concept:
 - Identify *equivalent states* and merge them into a single state.
 - Can lead to more compact representation.
- State Equivalence:
 - Two states S_i and S_j of a FSM are said to be *equivalent* iff for every single input sequence X , the outputs are the same and the next states are equivalent:

$$\lambda(S_i, X) = \lambda(S_j, X)$$

$$\delta(S_i, X) \equiv \delta(S_j, X)$$
 where $\lambda(S_i, X)$ is the output for the present state S_i and input X , and $\delta(S_i, X)$ is the corresponding next state.

Switching Circuits and Logic Design (CS20202)

47

47

Minimization Using Implication Chart

1. Construct an *Implication Chart* that contains a square for every pair of states.
2. Compare each pair of rows in the *State Table*. If the outputs associated with states i and j are different, place an 'x' in square $i-j$ to indicate i and j are *not equivalent*.
 - If the outputs are the same, place the *Implied Pairs* in square $i-j$. (If the next states of i and j are m and n respectively for some input X , then $m-n$ is an implied pair)
 - If the outputs and the next states are the same (or if $i-j$ only implies itself), place a check (✓) in square $i-j$ to indicate i and j are *equivalent*.
3. Go through the table square by square. If square $i-j$ contains the implied pair $m-n$, and square $m-n$ contains a 'x', then i and j are *not equivalent*, and place a 'x' in square $i-j$.
4. If any 'x' is added in Step 3, repeat Step 3 until no more 'x' can be added.
5. For each square $i-j$ that does not contain 'x', the states i and j are *equivalent*.

Switching Circuits and Logic Design (CS20202)

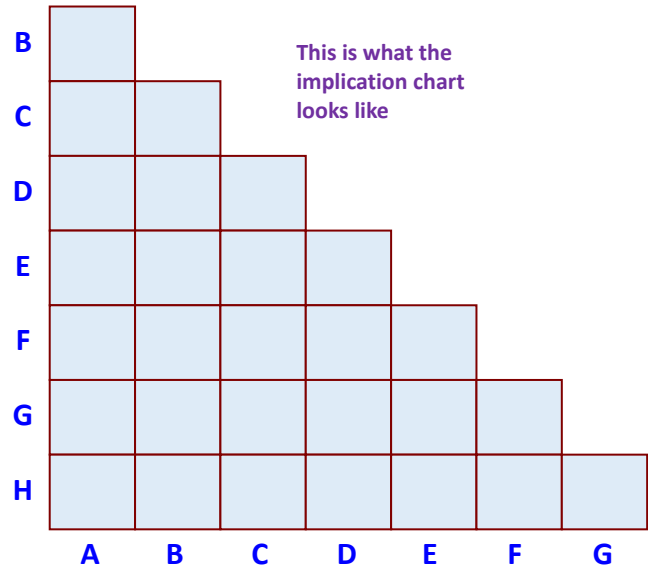
48

48

Example 1: A Moore Machine

State Table

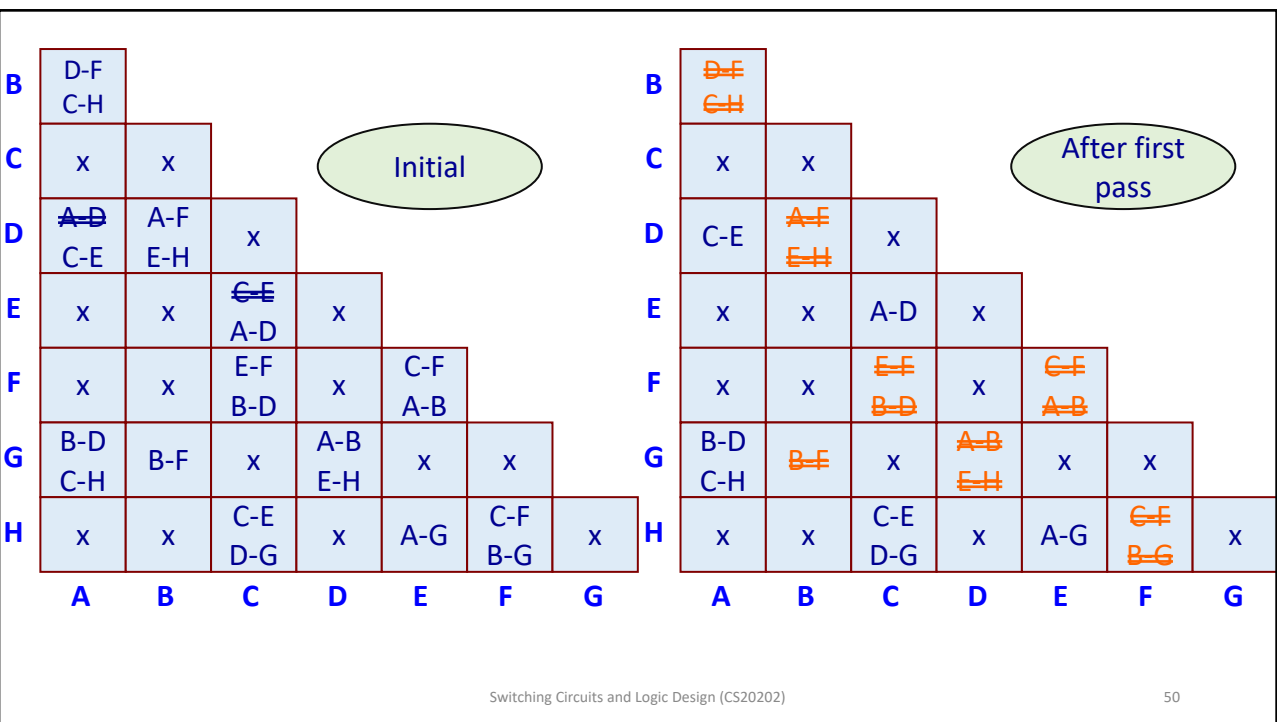
PS	NS		Output z
	X=0	X=1	
A	D	C	0
B	F	H	0
C	E	D	1
D	A	E	0
E	C	A	1
F	F	B	1
G	B	H	0
H	C	G	1



Switching Circuits and Logic Design (CS20202)

49

49



Switching Circuits and Logic Design (CS20202)

50

50

After second pass

B	B-E C-H					
C	x	x				
D	C-E	A-F E-H	x			
E	x	x	A-D	x		
F	x	x	E-F B-D	x	C-F A-B	
G	B-D C-H	B-E	x	A-B E-H	x	x
H	x	x	C-E D-G	x	A-G B-G	x
	A	B	C	D	E	F

$C \equiv E$
 $A \equiv D$

Reduced State Table

PS	NS		Output z
	X=0	X=1	
A	A	C	0
B	F	H	0
C	C	A	1
F	F	B	1
G	B	H	0
H	C	G	1

Switching Circuits and Logic Design (CS20202) 51

51

Example 2: A Mealy Machine

State Table

PS	NS, z	
	X=0	X=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

After first pass

B	x			
C	B-D	x		
D	x	✓	x	
E	C-E	x	C-E	x
F	x	B-E C-D	x	B-F B-C
	A	B	C	D

After second pass

B	x			
C	B-D	x		
D	x	✓	x	
E	C-E	x	B-F	x
F	x	C-D	x	B-F B-C
	A	B	C	D

Switching Circuits and Logic Design (CS20202) 52

52

State Table

PS	NS, z	
	X=0	X=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

Diagram showing state transitions for the initial state table. States are represented by letters A-F. Transitions are shown as arrows between boxes. A checkmark is in the box for state D when X=0.

Diagram showing state transitions after the third pass. The transition from state E to state F is crossed out with an orange X. A callout bubble says "After third pass".

Switching Circuits and Logic Design (CS20202) 53

53

State Table

PS	NS, z	
	X=0	X=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

Reduced State Table

PS	NS, z	
	X=0	X=1
A	E, 0	B, 1
B	F, 0	B, 0
E	A, 0	F, 1
F	B, 0	A, 0

Diagram showing state transitions for the reduced state table. States are represented by letters A-F. Transitions are shown as arrows between boxes. A checkmark is in the box for state C when X=0.

Switching Circuits and Logic Design (CS20202) 54

54

Equivalent FSMs

- Two finite state machines $N1$ and $N2$ are said to be *equivalent* if for every state p in $N1$, there is a state q in $N2$ such that p and q are equivalent; and conversely, for each state s in $N2$, there is a state t in $N1$, such that s and t are equivalent.
- An approach similar to implication tables can be used to verify equivalence.

Switching Circuits and Logic Design (CS20202)

55

55

An Example

- Consider two FSMs $N1$ and $N2$, whose reduced state tables are shown.
- We shall create a 4 x 4 implication chart to test state equivalence.

PS	NS, z	
	X=0	X=1
A	B, 0	A, 0
B	C, 0	D, 1
C	A, 0	C, 1
D	C, 0	B, 0

N1

PS	NS, z	
	X=0	X=1
S0	S3, 0	S1, 1
S1	S3, 0	S0, 0
S2	S0, 0	S2, 0
S3	S2, 0	S3, 1

N2

Switching Circuits and Logic Design (CS20202)

56

56

PS	NS, z		PS	NS, z	
	X=0	X=1		X=0	X=1
A	B, 0	A, 0	S0	S3, 0	S1, 1
B	C, 0	D, 1	S1	S3, 0	S0, 0
C	A, 0	C, 1	S2	S0, 0	S2, 0
D	C, 0	B, 0	S3	S2, 0	S3, 1

Equivalent states:

A – S2

B – S0

C – S3

D – S1

S0	x	C-S3 D-S1	A-S3 C-S1	x
S1	B-S3 A-S0	x	x	C-S3 B-S0
S2	B-S0 A-S2	x	x	C-S0 B-S2
S3	x	C-S2 D-S3	A-S2 C-S3	x
	A	B	C	D

S0	x	C-S3 D-S1	A-S3 C-S1	x
S1	B-S3 A-S0	x	x	C-S3 B-S0
S2	B-S0 A-S2	x	x	C-S0 B-S2
S3	x	C-S2 D-S3	A-S2 C-S3	x
	A	B	C	D

57

57

Another Example

PS	NS, z	
	X=0	X=1
S0	S5, 0	S1, 0
S1	S5, 0	S6, 0
S2	S2, 0	S6, 0
S3	S0, 1	S1, 0
S4	S4, 0	S3, 0
S5	S0, 0	S1, 0
S6	S5, 1	S1, 0

Machine N1

PS	NS, z	
	X=0	X=1
A	A, 0	B, 0
B	A, 0	C, 0
C	A, 1	B, 0

Machine N2

Switching Circuits and Logic Design (CS20202)

58

58

Incompletely Specified Machines

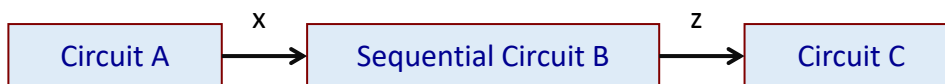
- The number of input combinations that can be applied may be a subset of all possible input combinations.
 - Other input combinations may be treated as don't cares.
- A FSM where the next state and output are not specified for all input combinations, is said to be *incompletely specified*.
 - We can extend the approach already discussed by adding don't cares to take care of this situation.

Switching Circuits and Logic Design (CS20202)

59

59

An Example

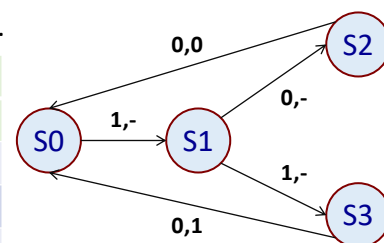


- Assume that circuit A can only generate two possible output sequences, $x = 100$ and 110 .
- After the sequence is received, the output of B will be $z = 0$ if 100 was received, and $z = 1$ if 110 was received.
 - Assume that circuit C ignores the value of z at all other times.

	t_0	t_1	t_2		t_0	t_1	t_2
$x =$	1	0	0	$z =$	-	-	0
	1	1	0		-	-	1

Possible input/output sequences

PS	NS, z	
	$x = 0$	$x = 1$
S0	-, -	S1, -
S1	S2, -	S3, -
S2	S0, 0	-, -
S3	S0, 1	-, -

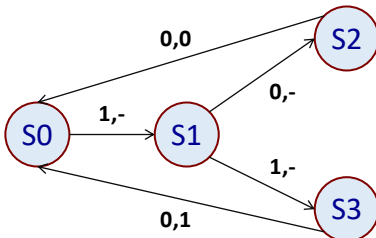


Switching Circuits and Logic Design (CS20202)

60

60

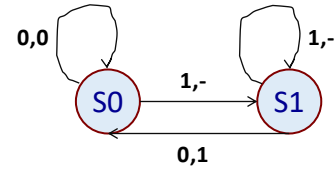
- We can fill up the don't cares suitably, so that the number of states can be reduced using row matching.



PS	NS, z	
	x = 0	x = 1
S0	S0, 0	S1, -
S1	S2, 1	S3, -
S2	S0, 0	S1, -
S3	S0, 1	S3, -

S1	x		
S2	✓	x	
S3	x	S2-S0	x
	S0	S1	S2

S0, S2 are equivalent
S1, S3 are equivalent



PS	NS, z	
	x = 0	x = 1
S0	S0, 0	S1, -
S1	S0, 1	S1, -

Switching Circuits and Logic Design (CS20202)

61

61

Design of Complete Systems – Data Path and Control Path

Switching Circuits and Logic Design (CS20202)

62

62

Concept of Data Path and Control Path

- For implementing complex systems, we typically break the design into two parts: (a) *Data Path* design, and (b) *Control Path* design.
- The *Data Path* contains the basic hardware units for performing calculation and storage (e.g. combinational circuits modules like adders, multipliers, comparators, multiplexers, etc. and sequential circuit elements like registers, counters, etc.).
- The *Control Path* is an FSM that generates the control signals for the Data Path in proper sequence to carry out the required operation.

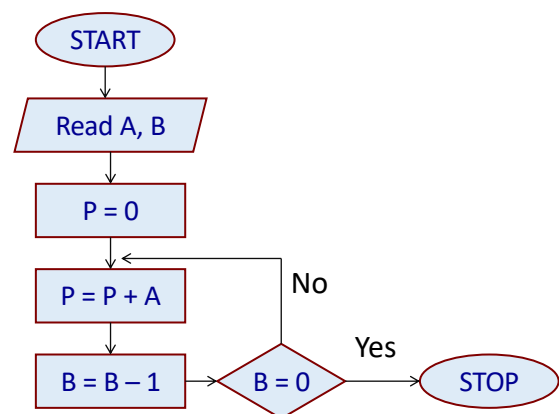
Switching Circuits and Logic Design (CS20202)

63

63

Example 1: Multiplication by Repeated Addition

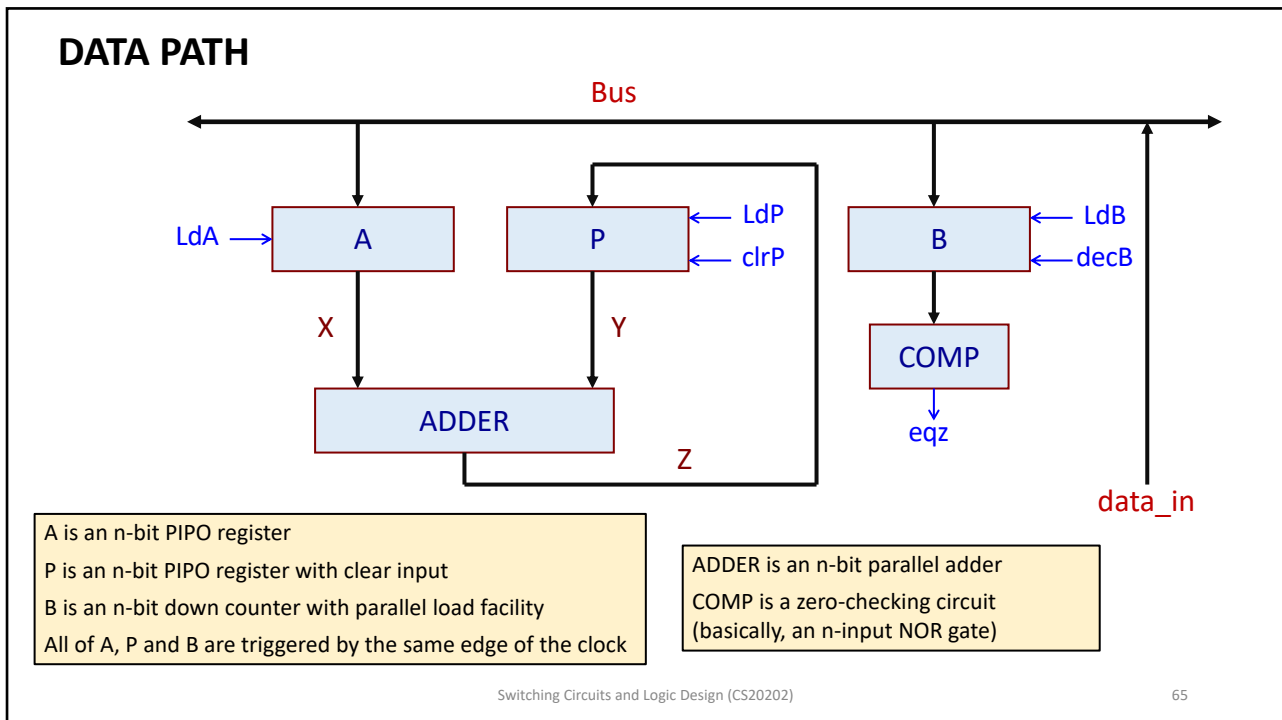
- We consider a simple algorithm using repeated addition.
 - Assume B is non-zero.
- We identify the functional blocks required in the data path, and the corresponding control signals.
- Then we design the FSM to implement the multiplication algorithm using the data path.



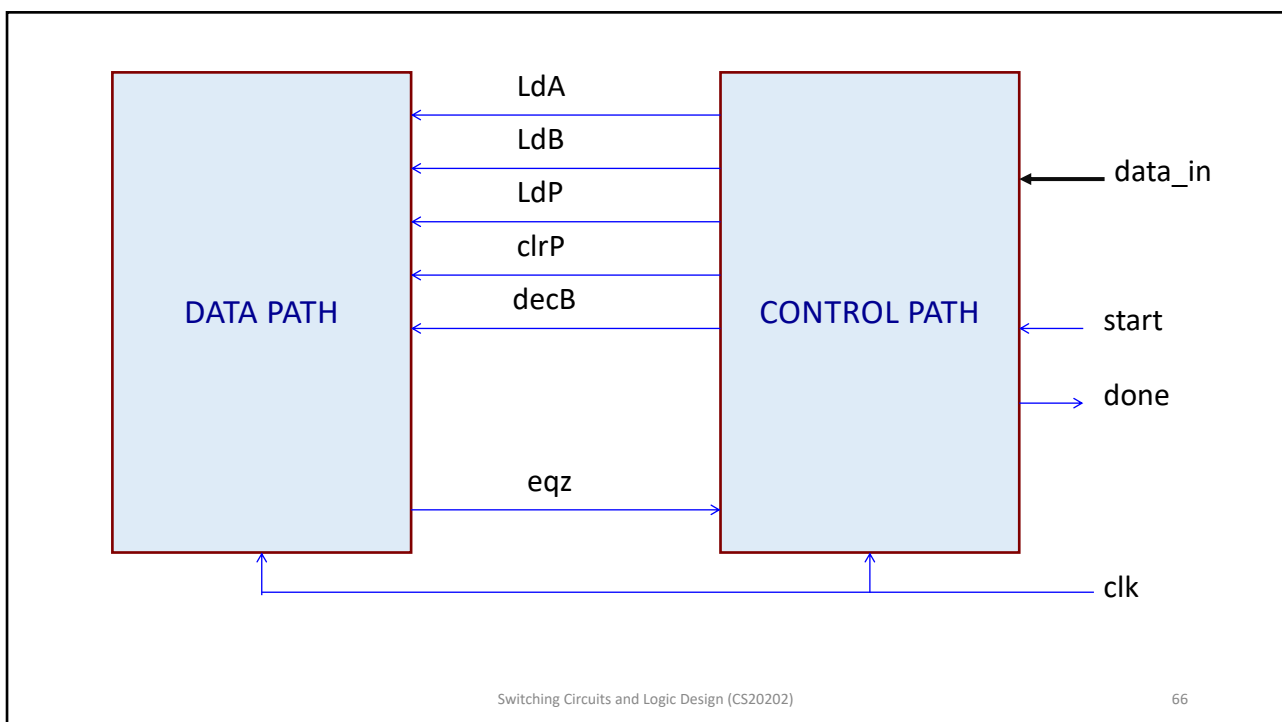
Switching Circuits and Logic Design (CS20202)

64

64

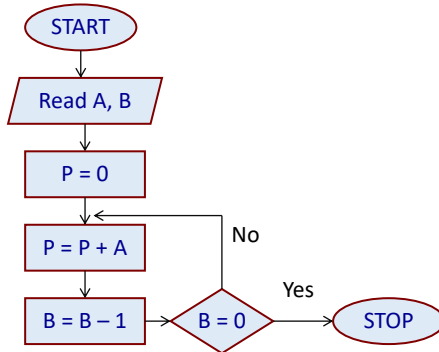


65

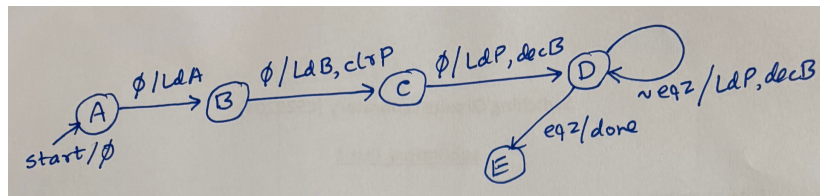
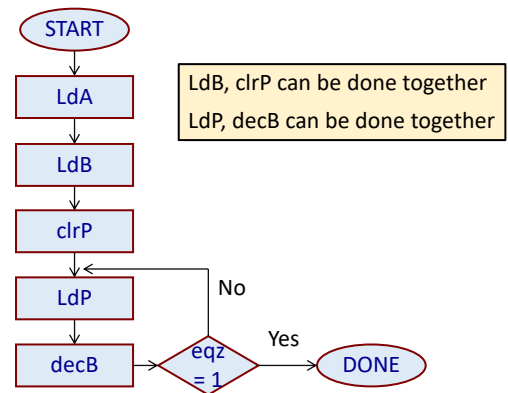


66

Design of the Control Path



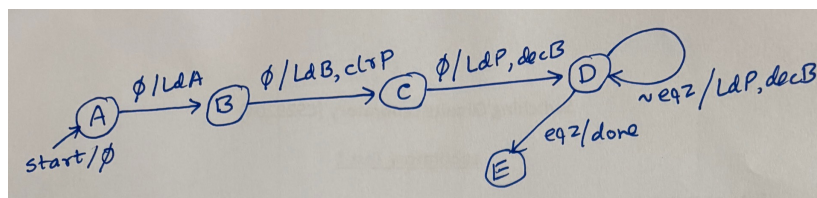
X/Y :: X indicates the input(s) that causes the transition; Y indicates the set of outputs that are set to 1 (remaining outputs are set to 0)



Switching Circuits and Logic Design (CS20202)

67

67



- Using FSM synthesis approach, we can realize the above state transition diagram using flip-flops and gates.
- "A" is the starting state; which will be reached whenever the "start" signal is active for the first time.

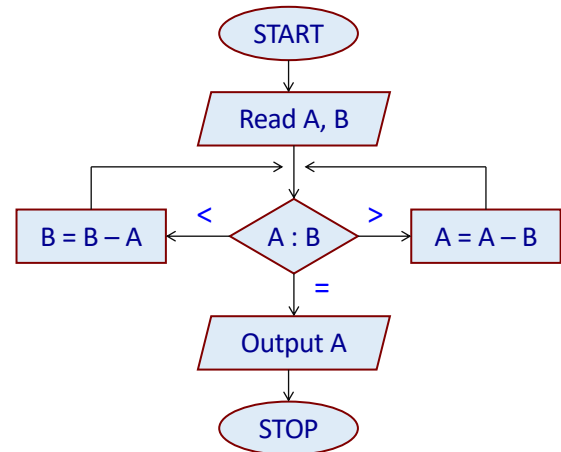
Switching Circuits and Logic Design (CS20202)

68

68

Example 2: GCD Computation by Repeated Subtraction

- We consider a simple algorithm using repeated subtraction.
- The two given numbers are A and B .
- We repeatedly keep on subtracting the smaller number from the larger number until they are equal.

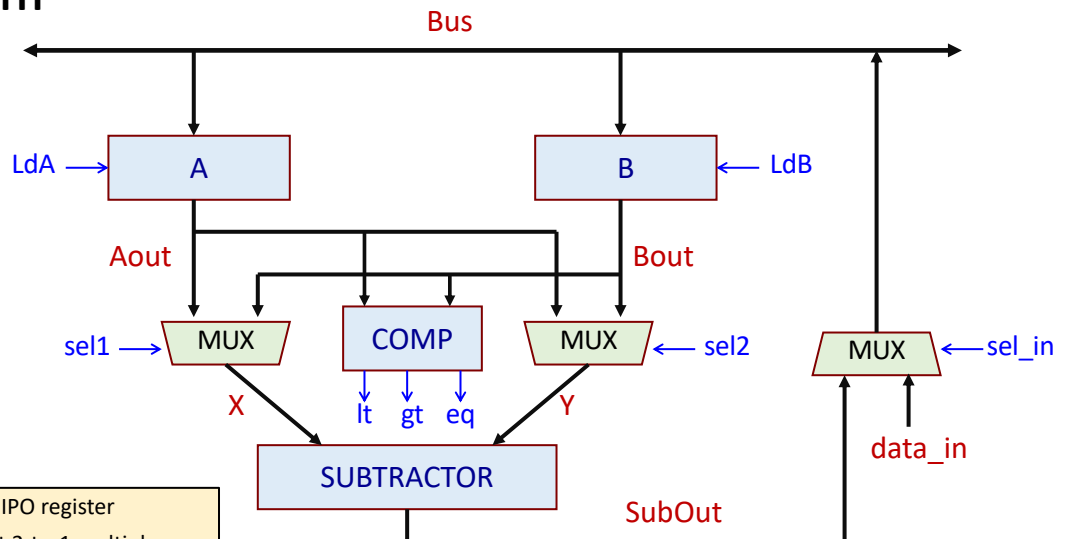


Switching Circuits and Logic Design (CS20202)

69

69

DATA PATH

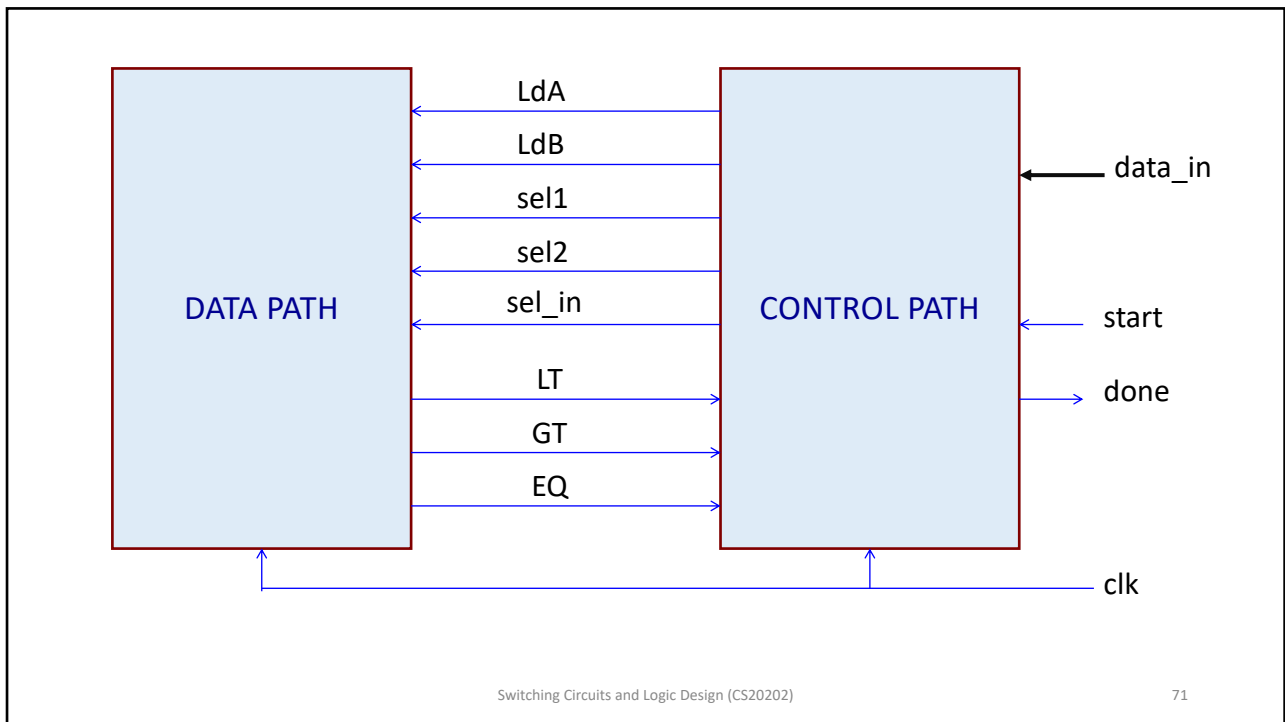


A, B are n-bit PIPO register
 MUX is an n-bit 2-to-1 multiplexer
 SUBTRACTOR is an n-bit subtractor
 COMP is an n-bit comparator

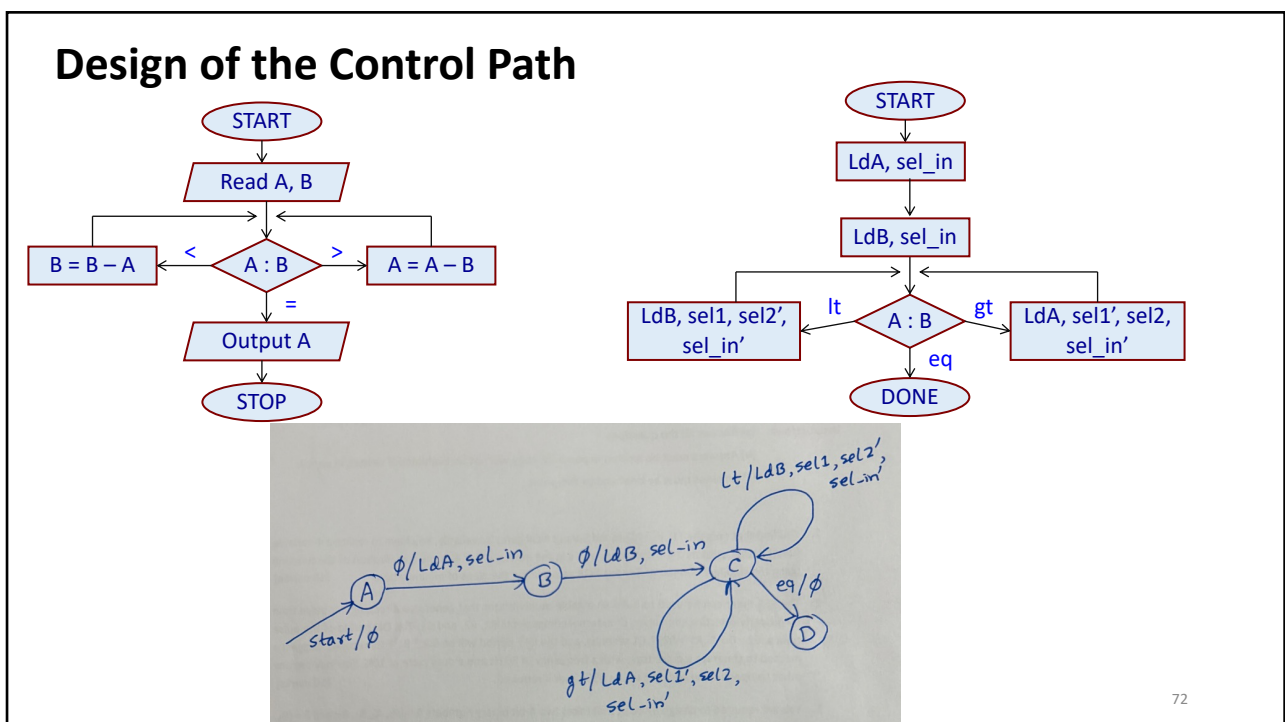
Switching Circuits and Logic Design (CS20202)

70

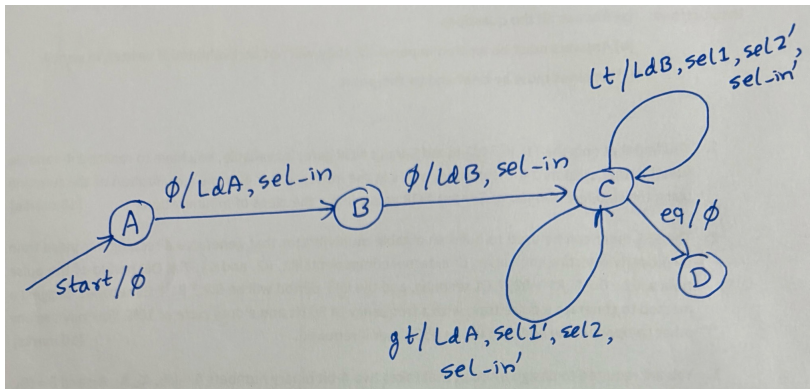
70



71



72



- Using FSM synthesis approach, we can realize the above state transition diagram using flip-flops and gates.
- “A” is the starting state; which will be reached whenever the “start” signal is active for the first time.